

# E01 — Matrix Sensing Benchmark: Muon vs SGD

## Overview

This experiment establishes the baseline comparison between **Muon** (with spectral normalization via exact SVD) and **SGD** (with momentum 0.9) on the **Matrix Sensing** problem. Matrix sensing is a fundamental low-rank matrix recovery problem where the goal is to recover a rank- $r$  matrix  $X^* \in \mathbb{R}^{d \times d}$  from  $m$  linear measurements.

**Why this matters:** Matrix sensing provides a clean optimization landscape (under RIP conditions) where both algorithms converge to the global optimum. This comparison reveals whether Muon's spectral normalization provides practical convergence advantages.

**Experiment ID:** E01 | **Problem Domain:** Matrix Sensing | **Algorithms:** Muon-Exact vs SGD

## Scientific Question

### Hypothesis

Muon's spectral normalization achieves faster convergence (lower  $K_\epsilon$ ) than SGD on matrix sensing, by equalizing gradient directions across singular values.

### Key Metrics

- $K_\epsilon$ : Iterations to reach  $\epsilon = 0.01$
- **min\_loss**: Best loss achieved
- **final\_loss**: Loss after 2000 iterations
- **time\_s**: Wall-clock time
- $F_\epsilon$ : Total FLOPs to convergence

### Statistical Framework

Paired t-tests per dimension with Cohen's  $d$  effect size.

## Experimental Design

| Parameter  | Value               | Description                   |
|------------|---------------------|-------------------------------|
| $d$        | {50, 60, 70}        | Matrix dimension              |
| $r$        | 5                   | Target rank of $X^*$          |
| $m$        | $2 \cdot d \cdot r$ | Measurements (RIP bound)      |
| lr         | 0.01                | Learning rate                 |
| noise      | 0.0                 | No observation noise          |
| dist       | normal              | Gaussian measurement matrices |
| spectrum   | hard-cutoff         | Flat top- $r$ singular values |
| $\kappa$   | 1.0                 | Condition parameter           |
| init_scale | 0.01                | Std dev of initialization     |
| Iterations | 2000                | Max iterations per run        |
| Seeds      | 10                  | Independent replications      |
| $\epsilon$ | 0.01                | Convergence threshold         |

**Total runs:** 2 algorithms x 3 dimensions x 10 seeds = 60 runs

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

plt.style.use('seaborn-v0_8-whitegrid')

df = pd.read_csv('../results_v3/E01_detailed_results.csv')
print(f"Shape: {df.shape}")
print(f"Algorithms: {df['algo'].unique()}")
print(f"Dimensions: {sorted(df['d'].unique())}")
print(f"Seeds: {sorted(df['seed'].unique())}")
df.head()
```

## Exploratory Data Analysis

Summary statistics grouped by algorithm and dimension.

```
In [ ]: summary = df.groupby(['algo', 'd']).agg({
    'K_epsilon': ['mean', 'std', 'min', 'max'],
    'min_loss': ['mean', 'std'],
    'final_loss': ['mean', 'std'],
    'time_s': ['mean', 'std'],
    'F_eps': ['mean', 'std']
}).round(4)
print("Summary by (algo, d):")
```

```
print(summary)

print("\nConvergence rates:")
conv = df.groupby(['algo', 'd'])['I_conv'].mean()
print(conv)
```

## Comparative Analysis: Paired t-tests

For each dimension, we compare  $K_\epsilon$  between Muon and SGD using paired t-tests on matched seeds.

```
In [ ]: print("Paired t-tests: Muon vs SGD (K_epsilon)")
print("-" * 75)
results = []
for d in sorted(df['d'].unique()):
    muon_k = df[(df['algo'] == 'Muon-Exact') & (df['d'] == d)].sort_values('I_conv')
    sgd_k = df[(df['algo'] == 'SGD') & (df['d'] == d)].sort_values('seed')['I_conv']
    diff = muon_k - sgd_k
    t_stat, p_val = stats.ttest_rel(muon_k, sgd_k)
    cohens_d = diff.mean() / diff.std(ddof=1) if diff.std(ddof=1) > 0 else 0
    sig = "****" if p_val < 0.001 else "***" if p_val < 0.01 else "*" if p_val < 0.05 else ""
    print(f"d={d:>3d}: Muon={muon_k.mean():>6.1f} +/- {muon_k.std():>4.1f} | "
          f"SGD={sgd_k.mean():>6.1f} +/- {sgd_k.std():>4.1f} | "
          f"diff={diff.mean():>6.1f} | t={t_stat:>6.3f} | p={p_val:.4f} {sig}")
    results.append({'d': d, 'muon_mean': muon_k.mean(), 'sgd_mean': sgd_k.mean(),
                   'diff': diff.mean(), 't_stat': t_stat, 'p_value': p_val, 'sig': sig})
import pandas as pd
results_df = pd.DataFrame(results)
print("Results table:")
print(results_df)
```

## Visualization 1: Bar Chart of Average $K_\epsilon$ by Dimension

```
In [ ]: fig, ax = plt.subplots(figsize=(10, 6))
dims = sorted(df['d'].unique())
x = np.arange(len(dims)); width = 0.35

muon_means = [df[(df['algo'] == 'Muon-Exact') & (df['d'] == d)]['K_epsilon'].mean() for d in dims]
muon_stds = [df[(df['algo'] == 'Muon-Exact') & (df['d'] == d)]['K_epsilon'].std() for d in dims]
sgd_means = [df[(df['algo'] == 'SGD') & (df['d'] == d)]['K_epsilon'].mean() for d in dims]
sgd_stds = [df[(df['algo'] == 'SGD') & (df['d'] == d)]['K_epsilon'].std() for d in dims]

ax.bar(x - width/2, muon_means, width, yerr=muon_stds, label='Muon-Exact',
       color='#2E86AB', capsize=5, edgecolor='white', linewidth=1)
ax.bar(x + width/2, sgd_means, width, yerr=sgd_stds, label='SGD',
       color='#F18F01', capsize=5, edgecolor='white', linewidth=1)

ax.set_xlabel('Matrix Dimension $d$', fontsize=12)
ax.set_ylabel('Average $K_\epsilon$ (iterations to $\epsilon=0.01$)', fontsize=12)
```

```

ax.set_title('E01: Convergence Speed – Muon vs SGD', fontsize=14, fontweight
ax.set_xticks(x)
ax.set_xticklabels([f'$d$={d}' for d in dims])
ax.legend(fontsize=11)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('E01_K_epsilon_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

## Visualization 2: Minimum Loss Comparison (Log Scale)

```

In [ ]: fig, ax = plt.subplots(figsize=(10, 6))
for algo, color, off in [('Muon-Exact', '#2E86AB', -0.8), ('SGD', '#F18F01',
subset = df[df['algo'] == algo]
means = [subset[subset['d'] == d]['min_loss'].mean() for d in dims]
stds = [subset[subset['d'] == d]['min_loss'].std() for d in dims]
ax.errorbar([d + off for d in dims], means, yerr=stds, label=algo,
color=color, marker='o', markersize=10, capsize=5, linewidthth
ax.set_xlabel('Matrix Dimension $d$', fontsize=12)
ax.set_ylabel('Minimum Loss (log scale)', fontsize=12)
ax.set_title('E01: Minimum Loss Achieved', fontsize=14, fontweight='bold')
ax.set_yscale('log')
ax.legend(fontsize=11)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('E01_min_loss_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

## Visualization 3: Total FLOPs Comparison

```

In [ ]: fig, ax = plt.subplots(figsize=(10, 6))
muon_flops = [df[(df['algo']=='Muon-Exact') & (df['d']==d)]['F_eps'].mean() for d in
sgd_flops = [df[(df['algo']=='SGD') & (df['d']==d)]['F_eps'].mean() for d in

ax.bar(x - width/2, [f/1e9 for f in muon_flops], width, label='Muon-Exact',
ax.bar(x + width/2, [f/1e9 for f in sgd_flops], width, label='SGD', color='#

for i, d in enumerate(dims):
    ratio = muon_flops[i]/sgd_flops[i]
    ax.text(i, max(muon_flops[i], sgd_flops[i])/1e9 * 1.05,
            f'ratio: {ratio:.2f}', ha='center', fontsize=9, style='italic',

ax.set_xlabel('Matrix Dimension $d$', fontsize=12)
ax.set_ylabel('Total FLOPs ($\times 10^9$)', fontsize=12)
ax.set_title('E01: Computational Cost to Reach $\epsilon$', fontsize=14, for
ax.set_xticks(x); ax.set_xticklabels([f'$d$={d}' for d in dims])
ax.legend(); ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('E01_flops_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

## Detailed Statistical Tests

```
In [ ]: print("=" * 80)
print("Detailed Statistical Tests: Muon vs SGD")
print("=" * 80)
for d in sorted(df['d'].unique()):
    muon_d = df[(df['algo']=='Muon-Exact') & (df['d']==d)].sort_values('seed')
    sgd_d = df[(df['algo']=='SGD') & (df['d']==d)].sort_values('seed')
    muon_k, sgd_k = muon_d['K_epsilon'].values, sgd_d['K_epsilon'].values
    diff = muon_k - sgd_k
    t_k, p_k = stats.ttest_rel(muon_k, sgd_k)
    d_k = diff.mean()/diff.std(ddof=1) if diff.std(ddof=1)>0 else 0
    w_s, w_p = stats.wilcoxon(muon_k, sgd_k)
    print(f"\nd={d}:")
    print(f"  K_epsilon: Muon {muon_k.mean():.1f} +/- {muon_k.std():.1f} S")
    print(f"  Paired t-test: t={t_k:+.4f}, p={p_k:.6f}, Cohen's d={d_k:+.4f}")
    print(f"  Wilcoxon: W={w_s:.1f}, p={w_p:.6f}")
print("\nInterpretation: negative t = Muon faster; p<0.05 = significant")
```

## Conclusions & Interpretation

### Key Findings

1. **Convergence Speed ( $K_\epsilon$ ):** Muon achieves fewer iterations to reach  $\epsilon = 0.01$  than SGD across all dimensions ( $d \in \{50, 60, 70\}$ ). The differences are statistically significant with large effect sizes, confirming that spectral normalization accelerates early-to-mid convergence.
2. **Loss Quality:** SGD achieves substantially lower minimum loss values (often by orders of magnitude). Muon appears to plateau at a higher loss floor, likely because spectral normalization suppresses the small-gradient updates needed for fine-grained convergence.
3. **Computational Cost:** Despite fewer iterations, Muon requires more total FLOPs due to  $O(d^3)$  SVD overhead per step. The FLOPs ratio (Muon/SGD) grows with dimension.
4. **Trade-off:** Muon excels when moderate precision is sufficient; SGD excels for high-precision solutions. Algorithm selection should consider both iteration efficiency and per-step cost.